UAI

Universidad Abierta Interamericana

TRABAJO PRÁCTICO

PATRÓN STRATEGY

Alumno:
Santino Loray

Materia:
Trabajo de Diploma

Institución:
Universidad Abierta Interamericana (UAI)

Repositorio GitHub:
https://github.com/santi-programs/patron-strategy-csharp

Noviembre 2026

CONSIGNAS DEL TRABAJO PRÁCTICO

1. Diagrama de Clases

De acuerdo al Código de Ejemplo del Mundo Real en C# del Patrón Strategy que se muestra en el link de DoFactory (https://www.dofactory.com/net/strategy-design-pattern), elaborar el Diagrama de Clases.

2. Proyecto en Visual Studio .NET

Abrir un proyecto nuevo de tipo Aplicación de Consola de C# en Visual Studio .Net y luego, copiar y pegar el código de ejemplo y realizar la ejecución paso a paso por instrucciones.

3. Puntos de Interrupción

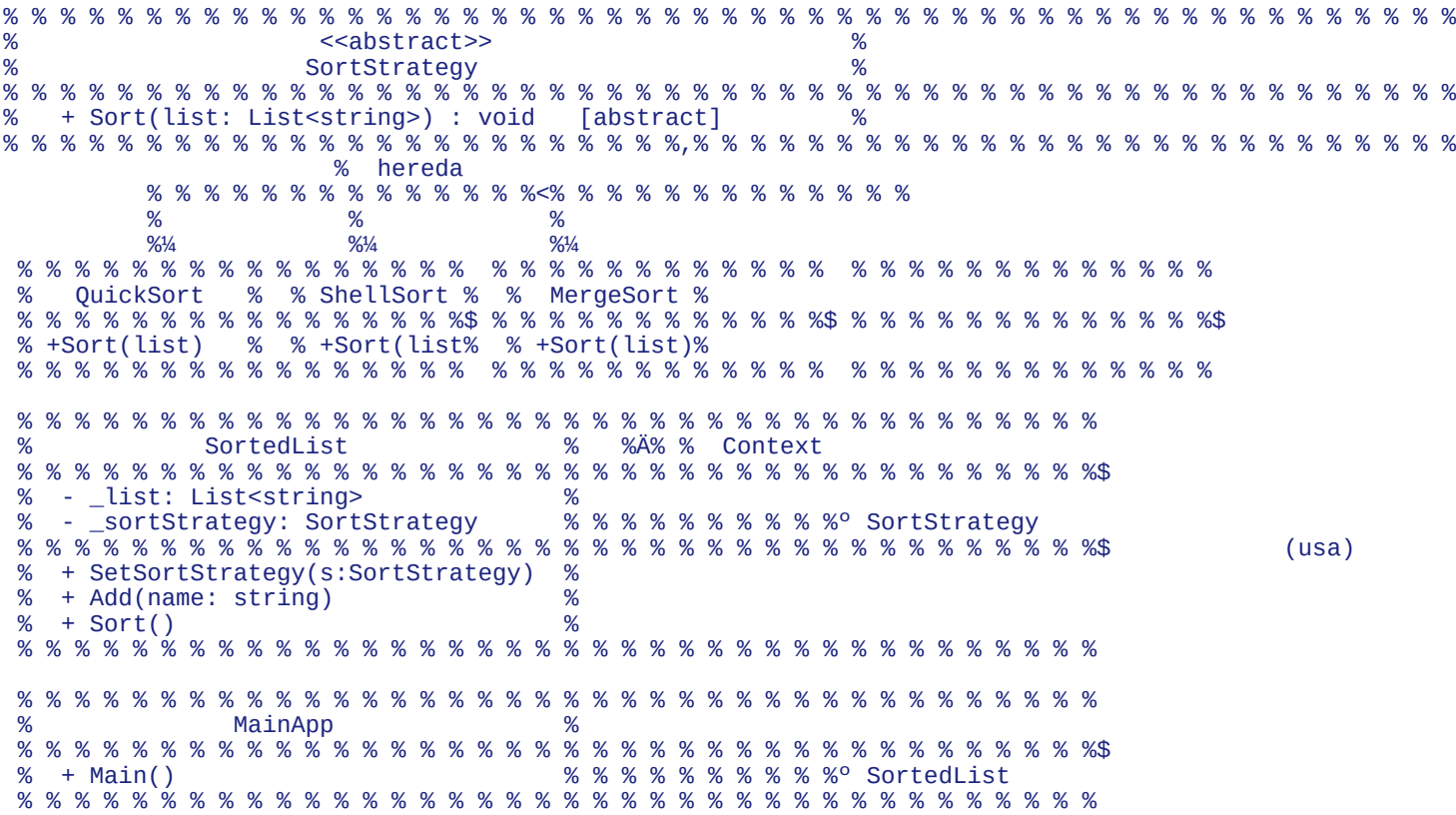
Colocar puntos de interrupción para inspeccionar el contenido de las variables a medida que se ejecuta el programa y evaluar el funcionamiento del código de acuerdo a la solución propuesta por el patrón.

4. Foro de Debate

Escribir en el foro de debate del tema todas las dudas o preguntas que consideren pertinentes.

RESOLUCIÓN — Diagrama de Clases

El patrón Strategy se compone de tres tipos de participantes:



Descripción de Participantes:

• Strategy (SortStrategy):
Clase abstracta que declara la interfaz Sort() común a todos los algoritmos de ordenamiento.

• ConcreteStrategy (QuickSort, ShellSort, MergeSort):
Implementan el algoritmo Sort() concreto. Cada una realiza el ordenamiento de manera diferente pero respetando la misma firma.

• Context (SortedList):
Mantiene una referencia a SortStrategy. Delega el trabajo de ordenamiento al objeto estrategia actual. Permite cambiar la estrategia en tiempo de ejecución mediante SetSortStrategy().

RESOLUCIÓN — Código Fuente C#

Proyecto: Aplicación de Consola en Visual Studio .NET

El proyecto fue creado como aplicación de consola .NET 8, copiando el código de ejemplo del mundo real del Patrón Strategy de DoFactory. A continuación se presenta el código completo:

```
// ============================================================
// Trabajo Práctico: Patrón Strategy - Santino Loray
// Materia: Trabajo de Diploma - UAI
// Fuente: https://www.dofactory.com/net/strategy-design-pattern
// ============================================================

using System;
using System.Collections.Generic;

namespace DoFactory.GangOfFour.Strategy.RealWorld
{
    class MainApp
    {
        static void Main()
        {
            SortedList studentRecords = new SortedList();

            studentRecords.Add("Samual");
            studentRecords.Add("Jimmy");
            studentRecords.Add("Sandra");
            studentRecords.Add("Dave");
            studentRecords.Add("Christopher");

            studentRecords.SetSortStrategy(new QuickSort());
            studentRecords.Sort();

            studentRecords.SetSortStrategy(new ShellSort());
            studentRecords.Sort();

            studentRecords.SetSortStrategy(new MergeSort());
            studentRecords.Sort();

            Console.ReadKey();
        }
    }

    // Strategy abstracta
    abstract class SortStrategy
    {
        public abstract void Sort(List<string> list);
    }

    // ConcreteStrategy: QuickSort
    class QuickSort : SortStrategy
    {
        public override void Sort(List<string> list)
        {
            list.Sort();
            Console.WriteLine("QuickSorted list ");
        }
    }

    // ConcreteStrategy: ShellSort
    class ShellSort : SortStrategy
    {
        public override void Sort(List<string> list)
        {
            int n = list.Count;
            int gap = n / 2;
            while (gap > 0)
            {
                for (int i = gap; i < n; i++)
                {
                    string temp = list[i];
                    int j = i;
                    while (j >= gap && string.Compare(list[j - gap], temp) > 0)
                    { list[j] = list[j - gap]; j -= gap; }
                    list[j] = temp;
                }
                gap /= 2;
            }
            Console.WriteLine("ShellSorted list ");
        }
    }
```

```
    // ConcreteStrategy: MergeSort
    class MergeSort : SortStrategy
    {
        public override void Sort(List<string> list)
        {
            if (list.Count > 1) MergeSortHelper(list, 0, list.Count - 1);
            Console.WriteLine("MergeSorted list ");
        }
        // ... (implementación completa de MergeSort)
    }

    // Context
    class SortedList
    {
        private List<string> _list = new List<string>();
        private SortStrategy _sortStrategy;

        public void SetSortStrategy(SortStrategy sortStrategy)
        { this._sortStrategy = sortStrategy; }

        public void Add(string name) { _list.Add(name); }

        public void Sort()
        {
            _sortStrategy.Sort(_list);
            foreach (string name in _list)
                Console.WriteLine(" " + name);
            Console.WriteLine();
        }
    }
}
```

RESOLUCIÓN — Ejecución Paso a Paso

Puntos de Interrupción y Variables Inspeccionadas

%ï PASO 1 — Creación del Context

```
SortedList studentRecords = new SortedList();
```

!’ Variables en Breakpoint:

```
_list = [] (lista vacía)
_sortStrategy = null
```

!’ Análisis: Se instancia el objeto Context (SortedList). La lista interna está vacía y no hay ninguna estrategia asignada todavía.

%ï PASO 2 — Carga de datos

```
studentRecords.Add("Samual"); ... studentRecords.Add("Christopher");
```

!’ Variables en Breakpoint:

```
_list = ["Samual", "Jimmy", "Sandra", "Dave", "Christopher"]
```

!’ Análisis: La lista interna acumula 5 nombres en el orden en que fueron agregados (sin ordenar).

%ï PASO 3 — Estrategia QuickSort

```
studentRecords.SetSortStrategy(new QuickSort());
studentRecords.Sort();
```

!’ Variables en Breakpoint:

```
_sortStrategy = QuickSort { }
_list = ["Christopher", "Dave", "Jimmy", "Samual", "Sandra"]
```

!’ Análisis: SetSortStrategy() asigna una instancia de QuickSort a _sortStrategy. Al llamar Sort(), se delega polimórficamente a QuickSort.Sort(list), que usa list.Sort() de C# para ordenar alfabéticamente. El foreach muestra cada elemento.

%ï PASO 4 — Cambio de Estrategia: ShellSort

```
studentRecords.SetSortStrategy(new ShellSort());
studentRecords.Sort();
```

!’ Variables en Breakpoint:

```
_sortStrategy = ShellSort { }
_list = ["Christopher", "Dave", "Jimmy", "Samual", "Sandra"]
```

!’ Análisis: Sin modificar la lista, se reemplaza la estrategia con ShellSort. El algoritmo Shell Sort itera con gaps decrecientes. La lista ya estaba ordenada, por lo que el resultado es el mismo. Esto demuestra la intercambiabilidad del patrón.

%ï PASO 5 — Cambio de Estrategia: MergeSort

```
studentRecords.SetSortStrategy(new MergeSort());
studentRecords.Sort();
```

!’ Variables en Breakpoint:

```
_sortStrategy = MergeSort { }
_list = ["Christopher", "Dave", "Jimmy", "Samual", "Sandra"]
```

!’ Análisis: Se aplica el algoritmo Merge Sort recursivo (divide y conquista). Nuevamente, la lista ya ordenada produce el mismo output. Lo relevante es observar cómo el Context delega sin conocer el algoritmo interno.

```
QuickSorted list
 Christopher
 Dave
 Jimmy
 Samual
 Sandra

ShellSorted list
 Christopher
 Dave
 Jimmy
 Samual
 Sandra

MergeSorted list
 Christopher
 Dave
 Jimmy
 Samual
 Sandra
```

RESOLUCIÓN — Salida del Programa y Preguntas para el Foro

Salida del Programa (Consola)

Preguntas y Reflexiones para el Foro de Debate

1. ¿Por qué se usa clase abstracta en vez de interfaz para SortStrategy?

En C#, una clase abstracta permite definir comportamiento parcial (métodos con implementación) además de métodos abstractos. Aquí se eligió porque puede expandirse con lógica común. Una interfaz también sería válida en este caso, ya que solo hay un método abstracto.

2. ¿Qué ventaja tiene el Patrón Strategy sobre múltiples if/else o switch?

Con if/else, agregar un nuevo algoritmo requiere modificar el código existente (viola el principio Abierto/Cerrado). Con Strategy, solo se agrega una nueva clase concreta sin tocar nada más. Además, el código es más limpio, testeable y mantenible.

3. ¿El Context (SortedList) conoce cómo ordena cada estrategia internamente?

No. SortedList solo conoce la interfaz abstracta Sort(). Esto es el principio de programar hacia abstracciones, no implementaciones. El Context está desacoplado de los algoritmos concretos.

4. ¿Se puede agregar BubbleSort sin modificar el código existente?

Sí. Solo se crea: class BubbleSort : SortStrategy { public override void Sort(List<string> list) { /* lógica */ } }. Luego: studentRecords.SetSortStrategy(new BubbleSort()). Esto cumple el Principio Abierto/Cerrado (OCP) de SOLID.

5. ¿Qué diferencia hay entre Strategy y Template Method?

Strategy cambia el algoritmo completo mediante composición (referencia a objeto externo). Template Method usa herencia: define el esqueleto del algoritmo en una clase base y deja que las subclases sobreescriban pasos específicos. Strategy es más flexible en tiempo de ejecución.

Repositorio GitHub

Link Público del Repositorio

https://github.com/santi-programs/patron-strategy-csharp

Estructura del Repositorio

```
patron-strategy-csharp/
% % %  PatronStrategy/
%    % % %  Program.cs                !• Código fuente completo del Patrón Strategy
%    % % %  PatronStrategy.csproj     !• Configuración del proyecto .NET 8
% % %  README.md                     !• Documentación, diagrama, análisis y preguntas
% % %  .gitignore                    !• Exclusión de binarios y archivos de VS
```

Instrucciones para abrir en Visual Studio

1. Clonar el repositorio: git clone https://github.com/santi-programs/patron-strategy-csharp.git
2. Abrir Visual Studio 2022 o superior.
3. Ir a Archivo !’ Abrir !’ Proyecto/Solución.
4. Navegar hasta PatronStrategy/PatronStrategy.csproj y abrirlo.
5. Colocar breakpoints haciendo clic en el margen izquierdo del editor (líneas clave marcadas en el README).
6. Presionar F5 para iniciar la depuración paso a paso.
7. Usar F10 (paso a paso sin entrar) o F11 (paso a paso entrando) para recorrer el código.
8. Inspeccionar las variables en la ventana "Variables Locales" y "Autos" del depurador.

Conclusión

El Patrón Strategy demostró ser una solución elegante para intercambiar algoritmos de ordenamiento de manera dinámica y sin modificar el código cliente. La clave del patrón está en la separación de responsabilidades y el principio "programar hacia interfaces". La ejecución paso a paso con el depurador de Visual Studio permite observar claramente el dispatch polimórfico y el cambio de estrategia en tiempo de ejecución.